

Trajectory planning and feedforward design for electromechanical motion systems

Paul Lambrechts*, Matthijs Boerlage, Maarten Steinbuch

Control Systems Technology Group, Faculty of Mechanical Engineering, Eindhoven University of Technology, P.O. Box 513,
5600 MB Eindhoven, The Netherlands

Received 14 February 2003; accepted 28 February 2004

Abstract

This paper considers trajectory planning with given design constraints and design of a feedforward controller for single-axis motion control. A motivation is given for using fourth-order feedforward with fourth-order trajectories. An algorithm is given for calculating higher-order trajectories with bounds on all considered derivatives for point-to-point moves. It is shown that these trajectories are time-optimal in the most relevant cases. All required equations for fourth-order trajectory planning are derived. Implementation, discretization and quantization effects are considered. Simulations and hardware-in-the-loop experiments show superior effectiveness of fourth-order feedforward in comparison with lower-order feedforward.

© 2004 Elsevier Ltd. All rights reserved.

Keywords: Trajectory planning; Feedforward compensation; Motion; Point-to-point control; Industrial control; Numerical methods

1. Introduction

Feedforward control is a well-known technique for high-performance motion control problems as found in industry. It is, for instance, widely applied in robots, pick-and-place units and positioning systems. These systems are often embedded in a factory automation scheme, which provides desired motion tasks to the considered system. The current trend is to leave the details of planning and execution of the motion to the computer hardware dedicated to the control of the system: one or more motion controllers. The tasks of such a dedicated motion controller will then consist of:

- *trajectory planning*: the calculation of an allowable trajectory,
- *profile generation*: the representation of the trajectory in appropriate form (e.g. a time sequence with a given sample time),
- *feedforward control*: the calculation of input signals for actuation devices with the intention to obtain the trajectory,

- *system compensation*: to reduce or remove unwanted behaviour like measured disturbances or non-linearities,
- *feedback control*: the processing of available measurements and calculation of input signals for actuation devices to compensate for unknown disturbances and unmodelled behaviour,
- internal checks, diagnostics, safety issues, communication, etc.

This shows that the burden for the motion controller can be quite high, while usually also a high sampling rate is required to achieve the desired performance.

To simplify these tasks, trajectory planning, profile generation and feedforward control are usually done for each actuating device separately, relying on system compensation and feedback control to deal with interactions and non-linearities. In that case, each actuating device is considered to be acting on a simple object, usually a single mass, moving along a single degree of freedom. The feedforward control problem is then to generate the force required to perform the acceleration of the mass in accordance with the desired trajectory. Conversely, the desired trajectory should be such that the required force is allowable (in the sense of mechanical load on the system) and can be generated by

*Corresponding author. Tel.: +31-40-2472839; fax: +31-40-2461418.

E-mail address: p.f.lambrechts@tue.nl (P. Lambrechts).

Nomenclature			
$\bar{x}$	bound on $ x $	d	derivative of jerk (profile) (m/s ⁴ or rad/s ⁴)
$\hat{x}$	maximum value obtained by $ x $ if bound is not considered	F	actuator force, feedforward force (N) or torque (Nm)
x_0	initial value	m	mass (kg) or inertia (kgm ²)
$t_{\bar{x}}$	time interval during which $ x $ obtains its bound	c	spring stiffness (N/m or Nm/rad)
$t_i, i \in N$	switching time instances	k	viscous damping coefficient (Ns/m or Ns m/rad)
x	position, displacement (profile) (m or rad)	$q_{1...4}$	feedforward parameters
v	velocity (profile) (m/s or rad/s)	T_s	sampling time (s)
a	acceleration (profile) (m/s ² or rad/s ²)	s	Laplace transform variable
J	jerk (profile) (m/s ³ or rad/s ³)	z	shift operator

the actuating device. For obvious reasons, this approach is often referred to as ‘mass feedforward’ or ‘rigid body feedforward’. It allows a simple and practical implementation of trajectory planning and feedforward control.

The disadvantage of this approach is its dependence on feedback control to deal with unmodelled behaviour as mentioned before. The resulting problem formulation can be split into two.

- (1) During execution of the trajectory the position errors are large, such that feedback control actions are considerable. Actual velocity and acceleration (hence: actuator force) may therefore be much larger than planned. This may lead to undesired and even dangerous deviations from the planned trajectory and damage to actuator and system.
- (2) When arriving at the desired endpoint, the positioning error is large and the dynamical state of the controlled system is not settled. Although the trajectory has finished, it is often necessary to wait for a considerable time before the position error is settled within some given accuracy bounds before subsequent actions or motions are allowed. A practical consequence is the need for a complex test to determine whether settling has sufficiently occurred. Furthermore, it is a source of time uncertainty that may be undesirable on the factory automation level.

To improve on this, many academic and practical approaches are possible. These can roughly be categorized in three.

- (1) *Trajectory smoothing or shaping*: This can be done by simply reducing the acceleration and velocity bounds used for trajectory planning, but also by smoothing or shaping the trajectory and/or application of force (higher-order trajectories, S-curves, input shaping, filtering). The result of this can be very good, especially if the dynamical behaviour of the motion system is explicitly taken into account.

However, it may also lead to a considerable increase in execution time of the trajectory, often without a clear mechanism for finding a time optimal solution. Various examples of this approach can be found in Dijkstra, Rambaratsingh, Scherer, Bosgra, Steinbuch, and Kerssemakers (2000), Meckl, Arestides, and Woods (1998), Murphy and Watanabe (1992), Paganini and Giusto (1997) and Singer, Singhose, and Seering (1999).

- (2) *Feedforward control based on plant inversion*: This attempts to take the effect of unmodelled behaviour into account by either using a more detailed model of the motion system or by learning its behaviour based on measurements. An important practical disadvantage is that they do not provide an approach for designing an appropriate trajectory. Various examples of this can be found in Boerlage, Steinbuch, Lambrechts, and van de Wal (2003), Devasia (2000), Hunt, Meyer, and Su (1996), Park, Chang, and Lee (2001), Roover (1997), Roover and Sperling (1997), Tomizuka (1987), Torfs, Swevers, and De Schutter (1991) and Torfs, Vuerinckx, Swevers, and Schoukens (1998).
- (3) *Feedback control optimization (possibly aided by system compensation improvement)*: By improving the feedback controller, the positioning errors can be kept smaller during and at the end of the trajectory. Furthermore, settling will occur in a shorter time. Also in this case the design of an appropriate trajectory is not considered. Obviously, any feedback control design method can be used for this. Some references given above also include a discussion on the effect of feedback control on trajectory following e.g. see Roover (1997), Roover and Sperling (1997), Torfs et al. (1998).

This paper will provide a method for higher-order trajectory planning that can be used with all of the approaches given above. Furthermore, ‘fourth-order feedforward’ will be presented as a clear and well implementable extension of ‘rigid body feedforward’. It

will be shown that the combination of fourth-order trajectory planning and fourth-order feedforward is well matched with the general behaviour of electromechanical motion systems and obtains time optimality within given physical bounds (actuator device and motion system limits). Next, the effect of discrete time implementation will be considered: this includes the planning of a fourth-order trajectory in discrete time and the optimal compensation of time delays in the feedforward control signal. Finally, some simulation results and hardware-in-the-loop experiments are given to further motivate the use of fourth-order feedforward.

The next section will review rigid body feedforward, mostly with the purpose of introducing some notation (see also Nomenclature). Next, Section 3 will consider the extension of rigid body feedforward to fourth-order feedforward, based on an extended model of the motion system. A general algorithm for higher-order trajectory planning will be considered in Section 4, after which the relevant equations are derived for fourth-order trajectory planning in Section 5. Next, implementation aspects are considered in Section 6, followed by simulations and experiments in Section 7, and conclusions in Section 8.

2. Rigid body feedforward

The specifics of planning a trajectory and calculating a feedforward signal based on rigid body feedforward are fairly simple and can be found in many commercially available electromechanical motion control systems. In this section, a short review is given as an introduction to a standardized approach to higher-order feedforward calculations.

Consider the configuration of Fig. 1 with m denoting the mass of the motion system, F the force supplied by the actuating device, x the position and k a viscous damping term. The equation of motion for this simple configuration is of course

$$m\ddot{x} + k\dot{x} = F. \quad (1)$$

Now consider the planning of a point-to-point move for this system over a distance denoted as $\bar{x}$, while restricted by given bounds on maximal velocity $\bar{v}$ and maximal acceleration $\bar{a}$:

1. $t_{\bar{a}} = \sqrt{\bar{x}/\bar{a}}$, preliminary acceleration/deceleration time interval to obtain $\bar{x}$.
2. $\hat{v} = \bar{a} \cdot t_{\bar{a}}$, maximal velocity obtained during move.
3. $\hat{v} > \bar{v}$, test whether velocity violates bound:
 - if true: $t_{\bar{a}} = \bar{v}/\bar{a}$,
 - if false: no action.
4. $x_{\bar{a}} = \bar{a}t_{\bar{a}}^2$, distance covered during acceleration and deceleration.
5. $t_{\bar{v}} = (\bar{x} - x_{\bar{a}})/\bar{v}$, constant velocity time interval.

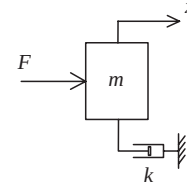


Fig. 1. Simple motion system: a single mass.

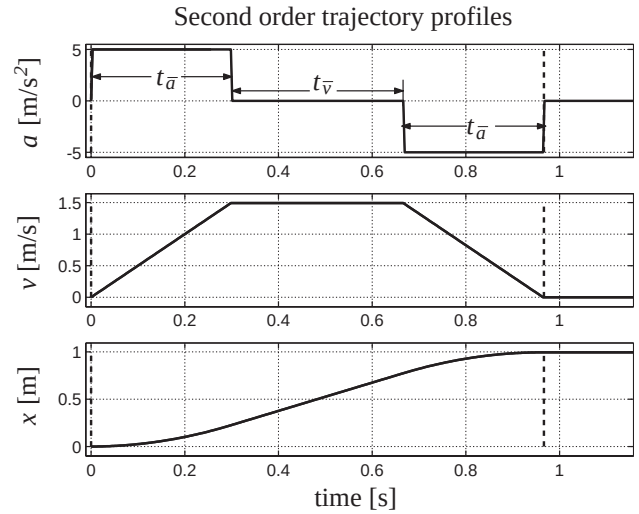


Fig. 2. Second-order trajectory determination.

Hence, this algorithm has calculated two time interval durations $t_{\bar{a}}$ and $t_{\bar{v}}$ such that

$$t_{\bar{x}} = 2t_{\bar{a}} + t_{\bar{v}}, \quad \bar{x} = \bar{a}t_{\bar{a}}^2 + \bar{v}t_{\bar{v}}. \quad (2)$$

Note that $t_{\bar{v}}$ automatically reverts to 0 if the velocity bound is not obtained. Construction of the acceleration profile a from $\bar{a}$, $t_{\bar{a}}$ and $t_{\bar{v}}$ is straightforward. From this, the desired trajectory can be determined by integrating it once to obtain the velocity profile v , and integrating it twice to obtain the position profile x ; see Fig. 2. As the position profile thus establishes the trajectory as a sequence of polynomials in time with a degree of at most 2, rigid body feedforward is also referred to as ‘second-order feedforward’.

The actual implementation of the trajectory planner and feedforward controller is indicated in Fig. 3. Note that the feedforward force F is simply calculated from Eq. (2) and the profiles in Fig. 2 as

$$F = ma + kv. \quad (3)$$

3. Higher-order feedforward

The previous section shows that rigid body feedforward is based on a simple single-mass model of the motion system. This implies that the performance of

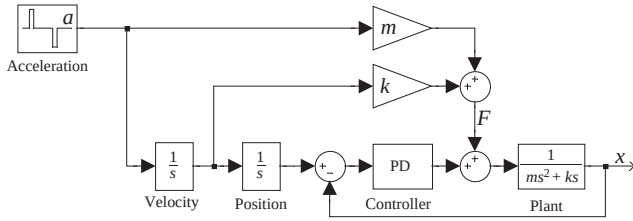


Fig. 3. Rigid body feedforward implementation.

rigid body feedforward is determined by how much the actual motion system deviates from this single-mass model. On the other hand, the performance of the motion system as a whole is also determined by the quality of system compensation and/or feedback control. It can be stated that the success of feedback control is such that in many cases the rigid body feedforward approach is considered sufficient, given that an appropriate feedback controller is required anyway for disturbance reduction and stabilization.

However, when considering further improvement of motion control system performance, the use of higher-order feedforward is often a very effective approach in comparison with improved feedback control. The first effect is that higher-order trajectories inherently have a lower energy content at higher frequencies, which results in a lower high-frequency content of the error signal, which in turn enables the feedback controller to be more effective. The second effect is that higher-order trajectories have less chance of demanding a motion which is physically impossible to perform by the given motion system, e.g. most power amplifiers exhibit a ‘rise time’ effect, such that it is impossible to produce a step-like change in force.

These effects are commonly referred to as ‘smoothing’ and result in a decrease of positioning errors during execution of the trajectory and a reduced settling time. The disadvantage of higher-order trajectories, i.e. the increase in trajectory execution time, is usually more than compensated by the reduced settling time. Because of this, many high-performance motion systems are already equipped with a third-order trajectory planner as a direct extension of rigid body feedforward. In this section, it will be determined that a fourth-order trajectory planner and feedforward calculation gives a significant further improvement.

The main argument is that an electromechanical motion system will usually have some compliance between actuator and load, and that both actuator and load will have a relevant mass. Support for this can for instance be found in Steinbuch and Norg (1998). Based on this, it is natural to extend the single mass model of Fig. 1 to the double-mass model of Fig. 4. Here m_1 denotes the mass of the actuator, m_2 the mass of the load, F the force supplied by the actuating device, x_1 the actuator position, x_2 the load position, c the stiffness

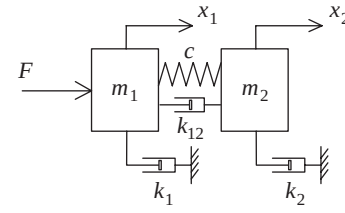


Fig. 4. Extended motion system: double mass.

between the two masses, k_{12} the viscous damping between the two masses, k_1 the viscous damping of the actuator towards ground and k_2 the viscous damping of the load towards ground. The equations of motion for this configuration are

$$\begin{aligned} m_1 \ddot{x}_1 &= -k_1 \dot{x}_1 - c(x_1 - x_2) - k_{12}(\dot{x}_1 - \dot{x}_2) + F, \\ m_2 \ddot{x}_2 &= -k_2 \dot{x}_2 + c(x_1 - x_2) + k_{12}(\dot{x}_1 - \dot{x}_2). \end{aligned} \quad (4)$$

Laplace transformation and substitution then results in the following expression:

$$F = \frac{q_1 s^4 + q_2 s^3 + q_3 s^2 + q_4 s}{k_{12} s + c} x_2 \quad (5)$$

with

$$\begin{aligned} q_1 &= m_1 m_2, \\ q_2 &= (m_1 + m_2) k_{12} + m_1 k_2 + m_2 k_1, \\ q_3 &= (m_1 + m_2) c + k_1 k_2 + (k_1 + k_2) k_{12}, \\ q_4 &= (k_1 + k_2) c. \end{aligned} \quad (6)$$

This implies that if some fourth-order trajectory has been planned for x_2 , from which the corresponding profiles for velocity v , acceleration a , jerk J and derivative of jerk d can be derived, the feedforward force F can be calculated as

$$F = \frac{1}{k_{12} s + c} \{q_1 d + q_2 J + q_3 a + q_4 v\}. \quad (7)$$

Analogous to the implementation given in Fig. 3, this feedforward scheme can readily be implemented as given in Fig. 5. Note that it is convenient to specify the trajectory by means of the derivative of jerk profile d , such that all other profiles can easily be obtained by integration. An algorithm for obtaining d will be the subject of the next sections.

4. Higher-order trajectory planning

Planners for second- and third-order trajectories are fairly well known in industry and academia and there are many approaches for obtaining a valid solution. Extension to fourth-order trajectory planning is however not trivial. In this section, the main objectives of trajectory planning, in general, will be reviewed and an algorithm will be set up for obtaining these objectives

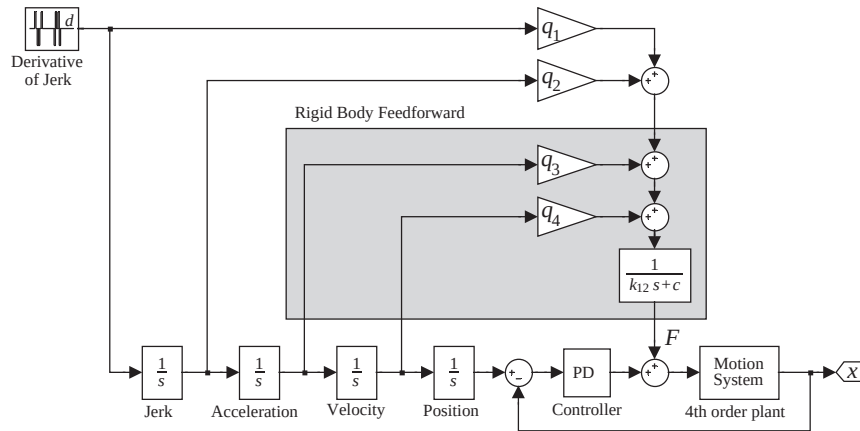


Fig. 5. Fourth-order feedforward implementation.

more or less irrespective of the order of the resulting trajectory.

A trajectory planner for a point-to-point move will be considered, although the resulting algorithm can be adjusted for relevant alternative objectives like for instance speed control instead of position control, scanning motions or speed change operations. The objectives that are important for applicability of any trajectory planning algorithm are given below:

- **Timing:** Time optimality should be obtained; at least it must be clear what the consequences are for the trajectory execution time when considering the effect of boundary conditions.
- **Actuator effort:** This is usually the basis for the selection of bounds for velocity, acceleration and jerk, although they may also be related to bounds on mechanics or safety issues.
- **Accuracy:** For point-to-point moves, the planned end position of the trajectory must be equal to the desired end position (within measurement accuracy).
- **Complexity:** Usually, trajectory planning is thought of as being done off-line. In practice, however, the desired end position often becomes available at the moment the trajectory should start. Hence, the time required for planning the trajectory is lost and should be minimized.
- **Reliability:** The planning algorithm should always come up with a valid solution.
- **Implementation:** Trajectory planning is done in computer hardware, and is therefore subject to discretization and digitization.

These objectives, except the final one, are all met by the simple algorithm given in Section 2. Obviously, timing is minimal if the bound $\bar{v}$ is discarded and only acceleration occurs at the maximal allowable $\bar{a}$. Introducing $\bar{v}$ always leads to a reduction of $t_{\bar{a}}$ (easily verified). The result will be that $v = \bar{v}$ is obtained in the shortest possible time, and is continued for as long as possible. Hence, the trajectory execution time is always

minimal, given the bounds $\bar{a}$ and $\bar{v}$. Next, actuator effort is immediately taken into account by means of the given bounds, accuracy is precise, complexity is low and reliability is high (no iteration loops that can hang up, algorithm only fails if a non-positive bound is specified). The implementation problem is considered later.

Given the advantages of this algorithm, it is desirable to generalize it for planning higher-order trajectories. The position displacement $\bar{x}$ and bounds on all derivatives of the trajectory up to the highest-order derivative d (indicated as $\bar{d}$) are assumed to be given. Furthermore, all derivatives are required to be equal to zero at the start and end positions.

- (1) Determine a symmetrical trajectory that has d equal to either $\bar{d}$ or $-\bar{d}$ at all times and obtains displacement $\bar{x}$.
- (2) Determine $t_{\bar{d}}$: the shortest time that d remains constant (always the first period).
- (3) Calculate the maximal value of velocity $\hat{v}$ obtained during this trajectory.
- (4) Test $\hat{v} > \bar{v}$:
 - if true re-calculate $t_{\bar{d}}$ based on $\bar{v}$ ($t_{\bar{d}}$ decreases),
 - if false continue.
- (5) Calculate the maximal value of acceleration $\hat{a}$ obtained during this trajectory.
- (6) Test $\hat{a} > \bar{a}$:
 - if true re-calculate $t_{\bar{d}}$ based on $\bar{a}$ ($t_{\bar{d}}$ decreases),
 - if false continue.
- (7) Continue these tests and possible recalculations until d ; the last test is performed on j , defined as the highest derivative before d . The resulting $t_{\bar{d}}$ will not be changed anymore.
- (8) Extend the trajectory symmetrically with periods of constant j whenever j reaches the value $\bar{j}$ or $-\bar{j}$: this must be done such that the required displacement $\bar{x}$ is obtained.
- (9) Determine $t_{\bar{j}}$: the shortest time that j remains constant.

- (10) Starting with velocity and ending with the derivative before J , calculate the maximal value and recalculate t_j if the appropriate bound is violated (each re-calculation can only decrease t_j).
- (11) The resulting t_j will not be changed anymore.
- (12) Extend the trajectory symmetrically with periods of constant next lower derivative; again such that the required displacement $\bar{x}$ is obtained. Determine the associated time interval and do the tests and recalculations resulting in a final value for it.
- (13) Continue until v : the constant speed phase duration $t_{\bar{v}}$ is calculated such that the required displacement $\bar{x}$ is obtained (i.e. the displacement 'left to go' divided by $\bar{v}$).
- (14) Finished: $t_{\bar{a}}$, t_j , etc. until $t_{\bar{v}}$ completely determine the trajectory.

The properties of the resulting trajectories (in the sense of the objectives given above) appear to depend partly on the order of the trajectory planner. Obviously, the required calculations become more complex with increasing order. Time-optimality is still automatically obtained with third-order trajectories, but not necessarily for fourth-order trajectories (this can be obtained but costs much complexity at a small gain with respect to a good sub-optimal solution). In principle, higher than fourth-order trajectories can also be planned by means of this algorithm, but this is considered impractical due to the large increase in complexity. It is noted here that all calculations for third- and fourth-order planning can be done analytically with fairly basic mathematical functions.

5. Fourth-order trajectory planning

This section will provide the derivations and calculations necessary to 'fill in' the algorithm developed in the previous section for fourth-order trajectory planning. Again it will be assumed that a symmetrical trajectory must be planned for a point-to-point move over a distance $\bar{x}$. Bounds are defined on velocity ($\bar{v}$), acceleration ($\bar{a}$), jerk ($\bar{j}$) and derivative of jerk ($\bar{d}$). The trajectory planning algorithm will be based on the construction of a derivative of jerk profile that can be integrated four times to obtain the fourth-order position trajectory. A symmetrical trajectory is completely determined by four time intervals: the constant derivative of jerk interval $t_{\bar{d}}$, the constant jerk interval t_j , the constant acceleration interval $t_{\bar{a}}$ and the constant velocity interval $t_{\bar{v}}$. The resulting profiles are given in Fig. 6. Note that there are 16 time instances at which the derivative of jerk changes, including the starting time of the trajectory at t_0 .

For step 1 of the algorithm, the bounds on jerk, acceleration and velocity will be discarded, and only the

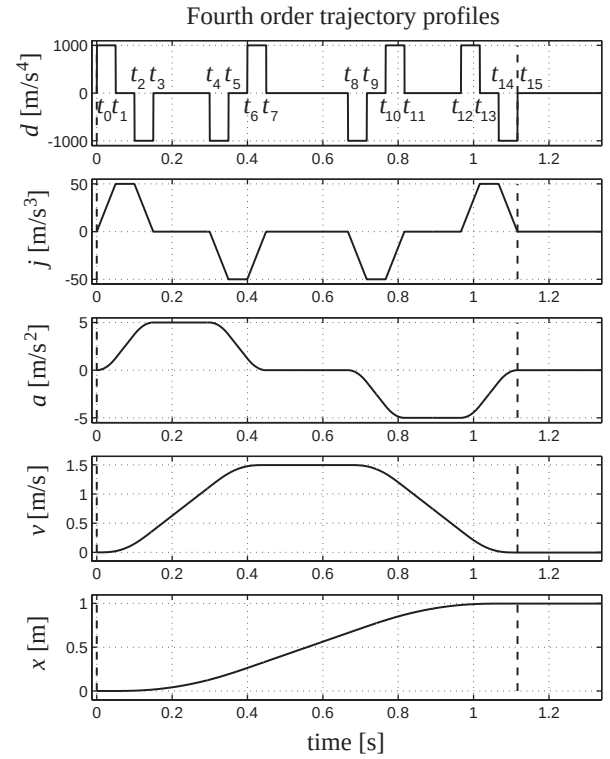


Fig. 6. Fourth-order trajectory planning.

bound on the derivative of jerk will be considered. This implies $t_{\bar{v}} = 0$, $t_{\bar{a}} = 0$ and $t_j = 0$ and it is clear that this will provide a lower bound for the trajectory execution time $t_{\bar{x}} = 8 \times t_{\bar{d}}$.

To obtain $t_{\bar{d}}$ (step 2), the relation between $t_{\bar{d}}$ and $\bar{x}$ with given $\bar{d}$ is needed. For this the constant value of derivative of jerk of $+\bar{d}$ or $-\bar{d}$ during each interval will be used. If the time instance of derivative of jerk change is set to 0, it is easily verified that for any time t during the constant derivative of jerk interval the fourth-order profiles for acceleration, velocity and position can be expressed as follows:

$$\begin{aligned}
 J(t) &= d_0 t + J_0, \\
 a(t) &= \frac{1}{2} d_0 t^2 + J_0 t + a_0, \\
 v(t) &= \frac{1}{6} d_0 t^3 + \frac{1}{2} J_0 t^2 + a_0 t + v_0, \\
 x(t) &= \frac{1}{24} d_0 t^4 + \frac{1}{6} J_0 t^3 + a_0 t^2 + v_0 t + x_0.
 \end{aligned} \tag{8}$$

Because the bounds on jerk, acceleration and velocity are assumed to be not violated, $t_j = 0$, $t_{\bar{a}} = 0$ and $t_{\bar{v}} = 0$, and consequently from Fig. 6: $t_2 = t_1 = t_{\bar{d}}$, $t_4 = t_3 = 2t_{\bar{d}}$, $t_6 = t_5 = 3t_{\bar{d}}$ and $t_8 = t_7 = 4t_{\bar{d}}$. Hence,

$$\begin{aligned}
 J(t_1) &= \bar{d} t_{\bar{d}}, \\
 a(t_1) &= \frac{1}{2} \bar{d} t_{\bar{d}}^2, \\
 v(t_1) &= \frac{1}{6} \bar{d} t_{\bar{d}}^3, \\
 x(t_1) &= \frac{1}{24} \bar{d} t_{\bar{d}}^4
 \end{aligned} \tag{9}$$

and for the next period in which $d = -\bar{d}$

$$\begin{aligned} J(t_3) &= -\bar{d}t_{\bar{d}} + J(t_1) = 0, \\ a(t_3) &= -\frac{1}{2}\bar{d}t_{\bar{d}}^2 + J(t_1)t_{\bar{d}} + a(t_1) = \bar{d}t_{\bar{d}}^2, \\ v(t_3) &= -\frac{1}{6}\bar{d}t_{\bar{d}}^3 + \frac{1}{2}J(t_1)t_{\bar{d}}^2 + a(t_1)t_{\bar{d}} + v(t_1) = \bar{d}t_{\bar{d}}^3, \\ x(t_3) &= -\frac{1}{24}\bar{d}t_{\bar{d}}^4 + \frac{1}{6}J(t_1)t_{\bar{d}}^3 + \frac{1}{2}a(t_1)t_{\bar{d}}^2 + v(t_1)t_{\bar{d}} + x(t_1) \\ &= \frac{7}{12}\bar{d}t_{\bar{d}}^4. \end{aligned} \quad (10)$$

Again using Eq. (8), the results of the subsequent periods can be calculated:

$$\begin{aligned} J(t_5) &= -\bar{d}t_{\bar{d}}, \quad J(t_7) = 0, \\ a(t_5) &= \frac{1}{2}\bar{d}t_{\bar{d}}^2, \quad a(t_7) = 0, \\ v(t_5) &= 1\frac{5}{6}\bar{d}t_{\bar{d}}^3, \quad v(t_7) = 2\bar{d}t_{\bar{d}}^3, \\ x(t_5) &= 2\frac{1}{24}\bar{d}t_{\bar{d}}^4, \quad x(t_7) = 4\bar{d}t_{\bar{d}}^4 \end{aligned} \quad (11)$$

and due to symmetry of the profile: $x(t_{15}) = 2x(t_7) = 8\bar{d}t_{\bar{d}}^4$. From this $t_{\bar{d}}$ can be calculated as required by steps 1 and 2 of the trajectory planning algorithm:

$$t_{\bar{d}} = \sqrt[4]{\frac{\bar{x}}{8\bar{d}}} \quad (12)$$

To continue with step 3, note that the maximal velocity is obtained at time instance t_7 . Hence, from Eq. (11) follows

$$\hat{v} = v(t_7) = 2\bar{d}t_{\bar{d}}^3. \quad (13)$$

If $\hat{v} > \bar{v}$ it is necessary to recalculate $t_{\bar{d}}$ as

$$t_{\bar{d}} = \sqrt[3]{\frac{\bar{v}}{2\bar{d}}}, \quad (14)$$

which establishes step 4. Next, step 5 follows from the calculation of maximal acceleration obtained at time instance t_3 . From Eq. (10) follows:

$$\hat{a} = a(t_3) = \bar{d}t_{\bar{d}}^2 \quad (15)$$

and if $\hat{a} > \bar{a}$ it is necessary to recalculate $t_{\bar{d}}$ as

$$t_{\bar{d}} = \sqrt{\frac{\bar{a}}{\bar{d}}}, \quad (16)$$

which establishes step 6. In this case, one further test is required (step 7) to do on the maximal jerk obtained at t_1 :

$$\hat{j} = J(t_1) = \bar{d}t_{\bar{d}} \quad (17)$$

and if $\hat{j} > \bar{j}$ it is necessary to recalculate $t_{\bar{d}}$ as

$$t_{\bar{d}} = \frac{\bar{j}}{\bar{d}} \quad (18)$$

Hence, this establishes a value for $t_{\bar{d}}$ that complies with all of the bounds $\bar{j}$, $\bar{a}$ and $\bar{v}$. From this point on, $t_{\bar{d}}$ can be considered a constant.

The next thing to do is to calculate t_j under the assumption that bounds $\bar{a}$ and $\bar{v}$ are not violated (steps 8 and 9). To do the necessary calculations, assume that $t_j > 0$, such that $t_2 > t_1$ and $t_6 > t_5$ (from Fig. 6). Eq. (8) and the fact that $d = 0$ during the period from t_1 to t_2

now implies

$$\begin{aligned} J(t_2) &= \bar{d}t_{\bar{d}}, \\ a(t_2) &= \frac{1}{2}\bar{d}t_{\bar{d}}^2 + \bar{d}t_{\bar{d}}t_j, \\ v(t_2) &= \frac{1}{6}\bar{d}t_{\bar{d}}^3 + \frac{1}{2}\bar{d}t_{\bar{d}}^2t_j + \frac{1}{2}\bar{d}t_{\bar{d}}t_j^2, \\ x(t_2) &= \frac{1}{24}\bar{d}t_{\bar{d}}^4 + \frac{1}{6}\bar{d}t_{\bar{d}}^3t_j + \frac{1}{4}\bar{d}t_{\bar{d}}^2t_j^2 + \frac{1}{6}\bar{d}t_{\bar{d}}t_j^3. \end{aligned} \quad (19)$$

Again using Eq. (8) it is possible to calculate jerk, acceleration, velocity and position at the consecutive time instances. Note, that maximal acceleration is obtained at t_3 and maximal velocity at t_7 . Now, it can be derived that the position obtained at the end of the move can be calculated as

$$x(t_{15}) = 8\bar{d}t_{\bar{d}}^4 + 16\bar{d}t_{\bar{d}}^3t_j + 10\bar{d}t_{\bar{d}}^2t_j^2 + 2\bar{d}t_{\bar{d}}t_j^3. \quad (20)$$

By setting $\bar{x} = x(t_{15})$, it is then possible to solve t_j from the following:

$$\begin{aligned} (2\bar{d}t_{\bar{d}}t_j^3 + (10\bar{d}t_{\bar{d}}^2)t_j^2 + (16\bar{d}t_{\bar{d}}^3)t_j + (8\bar{d}t_{\bar{d}}^4 - \bar{x}) &= 0 \\ \Rightarrow t_j^3 + (5t_{\bar{d}})t_j^2 + (8t_{\bar{d}}^2)t_j + \left(4t_{\bar{d}}^3 - \frac{\bar{x}}{2\bar{d}t_{\bar{d}}}\right) &= 0. \end{aligned} \quad (21)$$

This is a third-order polynomial equation in t_j , which has one positive real root that can be calculated as follows (see Bronshtein & Semendyayev, 1973):

$$\begin{aligned} p &:= -\frac{1}{9}t_{\bar{d}}^2, \\ q &:= -\frac{1}{27}t_{\bar{d}}^3 - \frac{\bar{x}}{4\bar{d}t_{\bar{d}}}, \\ D &:= p^3 + q^2, \\ r &:= \sqrt[3]{-q + \sqrt{D}}, \\ t_j &= r - \frac{p}{r} - \frac{5}{3}t_{\bar{d}}. \end{aligned} \quad (22)$$

After this, step 10 of the algorithm introduces the velocity and acceleration bounds again; the maximal velocity is obtained at t_7 :

$$\hat{v} = v(t_7) = 2\bar{d}t_{\bar{d}}^3 + 3\bar{d}t_{\bar{d}}^2t_j + \bar{d}t_{\bar{d}}t_j^2. \quad (23)$$

If $\hat{v} > \bar{v}$ the bound is violated and it is necessary to recalculate t_j from the second-order polynomial

$$t_j^2 + 3t_{\bar{d}}t_j + 2t_{\bar{d}}^2 - \frac{\bar{v}}{\bar{d}t_{\bar{d}}} = 0. \quad (24)$$

The positive real solution for this is

$$t_j = -\frac{1}{2}t_{\bar{d}} + \sqrt{\frac{t_{\bar{d}}^2}{4} + \frac{\bar{v}}{\bar{d}t_{\bar{d}}}}. \quad (25)$$

Next, the maximal acceleration is obtained at t_3 :

$$\hat{a} = a(t_3) = \bar{d}t_{\bar{d}}^2 + \bar{d}t_{\bar{d}}t_j. \quad (26)$$

If $\hat{a} > \bar{a}$ it is necessary to recalculate t_j as

$$t_j = \frac{\bar{a}}{\bar{j}} - t_{\bar{d}} \quad (27)$$

and step 11 of the planning algorithm is established.

In accordance with step 12, there is a further extension of the trajectory required for the calculation of $t_{\bar{a}}$. To do the necessary calculations, assume that $t_{\bar{a}} > 0$, such that $t_4 > t_3$ (from Fig. 6). Eq. (8) and the fact that $d = 0$ and $J = 0$ during the period from t_3 to t_4 can now again be used to calculate jerk, acceleration, velocity and position at the consecutive time instances. This results in the following expression for the obtained end position at t_{15} :

$$x(t_{15}) = 8\bar{d}t_{\bar{a}}^4 + 16\bar{d}t_{\bar{a}}^3t_j + 10\bar{d}t_{\bar{a}}^2t_j^2 + 2\bar{d}t_{\bar{a}}t_j^3 + \bar{d}t_{\bar{a}}^2t_{\bar{a}}^2 + \bar{d}t_{\bar{a}}t_j^2t_{\bar{a}} + 6\bar{d}t_{\bar{a}}^3t_{\bar{a}} + 9\bar{d}t_{\bar{a}}^2t_jt_{\bar{a}} + 3\bar{d}t_{\bar{a}}t_j^2t_{\bar{a}}. \quad (28)$$

Now, given that $t_{\bar{d}}$ and t_j are already determined, $t_{\bar{a}}$ can be solved such that $x(t_{15}) = \bar{x}$ from the following polynomial equation:

$$\{\bar{d}t_{\bar{a}}^2 + \bar{d}t_{\bar{a}}t_j\}t_{\bar{a}}^2 + \{6\bar{d}t_{\bar{a}}^3 + 9\bar{d}t_{\bar{a}}^2t_j + 3\bar{d}t_{\bar{a}}t_j^2\}t_{\bar{a}} + \{8\bar{d}t_{\bar{a}}^4 + 16\bar{d}t_{\bar{a}}^3t_j + 10\bar{d}t_{\bar{a}}^2t_j^2 + 2\bar{d}t_{\bar{a}}t_j^3 - \bar{x}\} = 0. \quad (29)$$

Clearly, $t_{\bar{a}}$ must be the positive root of this second-order polynomial equation.

Now introduce the velocity bound again:

$$\hat{v} = v(t_7) = 2\bar{d}t_{\bar{a}}^3 + 3\bar{d}t_{\bar{a}}^2t_j + \bar{d}t_{\bar{a}}t_j^2 + \bar{d}t_{\bar{a}}^2t_{\bar{a}} + \bar{d}t_{\bar{a}}t_jt_{\bar{a}} \quad (30)$$

and if $\hat{v} > \bar{v}$ the bound is violated such that $t_{\bar{a}}$ must be recalculated as

$$t_{\bar{a}} = \frac{\bar{v} - 2\bar{d}t_{\bar{a}}^3 - 3\bar{d}t_{\bar{a}}^2t_j - \bar{d}t_{\bar{a}}t_j^2}{\bar{d}t_{\bar{a}}^2 + \bar{d}t_{\bar{a}}t_j}. \quad (31)$$

This determines $t_{\bar{d}}$, t_j and $t_{\bar{a}}$ under the restriction of the given bounds and, as the next derivative is v , this establishes step 12 of the algorithm.

Finally, step 13 will determine the constant velocity time interval $t_{\bar{v}}$ such that the required total displacement $\bar{x}$ is obtained. From the previous calculations follows that $t_{\bar{v}} = 0$ if $t_{\bar{a}}$ is according to Eq. (29). But if $t_{\bar{a}}$ is reduced according to Eq. (31), the result will be that $x(t_{15}) < \bar{x}$ (with $x(t_{15})$ recalculated according to Eq. (28)), and a constant velocity phase must be added to the trajectory such that $x(t_4) - x(t_3) = \bar{x} - x(t_{15})$. Hence, $t_{\bar{v}}$ can be calculated as

$$t_{\bar{v}} = \frac{\bar{x} - x(t_{15})}{\bar{v}}. \quad (32)$$

This completes the calculation of the characteristics of the symmetrical fourth-order profile for a given distance $\bar{x}$ and given bounds $\bar{v}$, $\bar{a}$, $\bar{j}$ and $\bar{d}$. Furthermore, if the trajectory contains a constant velocity phase (i.e. $t_{\bar{v}} > 0$), the trajectory is time-optimal. The total displacement $\bar{x}$ can be expressed as a function of $\bar{d}$ and the times $t_{\bar{d}}$, t_j , $t_{\bar{a}}$ and $t_{\bar{v}}$:

$$\begin{aligned} \bar{x} = & \bar{d}(t_{\bar{d}}^2t_{\bar{a}}^2 + t_{\bar{d}}t_jt_{\bar{a}}^2 + 6t_{\bar{d}}^3t_{\bar{a}} + 9t_{\bar{d}}^2t_jt_{\bar{a}} + 3t_{\bar{d}}t_j^2t_{\bar{a}} + 8t_{\bar{d}}^4 \\ & + 16t_{\bar{d}}^3t_j + 10t_{\bar{d}}^2t_j^2 + 2t_{\bar{d}}t_j^3 + 2t_{\bar{d}}^3t_{\bar{v}} + 3t_{\bar{d}}^2t_jt_{\bar{v}} \\ & + t_{\bar{d}}t_j^2t_{\bar{v}} + t_{\bar{d}}^2t_{\bar{a}}t_{\bar{v}} + t_{\bar{d}}t_jt_{\bar{a}}t_{\bar{v}}). \end{aligned} \quad (33)$$

6. Implementation aspects

6.1. Switching times

From the previous sections, it is clear that the derivations for especially fourth-order trajectory planning are quite elaborate. However, the calculations actually required for implementation of this planner are relatively straightforward. The algorithm consists of a combination of polynomial calculations with simple if–then–else tests for which most state-of-the-art motion control hardware has standard algorithms.

When considering implementation of the planned trajectory into a feedforward control scheme, it is important to consider the effect of discretization. This implies that the integrators and the feedforward filter as given in the diagram of Fig. 5 will all have to be implemented in discrete time at some given sampling time interval T_s . This also implies that the switching time instances of the planned trajectory must be synchronized with the sampling time instances, i.e. the time intervals $t_{\bar{v}}$, $t_{\bar{a}}$, etc. must be multiples of T_s .

To obtain this, it must be accepted that time-optimality as resulting from the continuous time calculations in the previous sections is lost. The calculated time intervals must be rounded-off towards a multiple of T_s . This must be done such that the given bounds are not violated, but at the same time approximated as closely as possible. The approach proposed here is to extend the algorithm of Section 5. After each calculation or re-calculation of a time interval, the result is rounded-off upward to the next multiple of T_s . Next, the maximal value of the highest derivative is calculated accordingly.

As an example, consider the first calculation of $t_{\bar{d}}$ (Eq. (12)). The rounded-off value for $t_{\bar{d}}$ can be calculated as

$$t_{\bar{d}}' = \text{ceil}\left(\frac{t_{\bar{d}}}{T_s}\right) \times T_s \quad (34)$$

with $\text{ceil}(\cdot)$ denoting the rounding off towards the next higher integer. From Eq. (12) a new value for $\bar{d}$ can then be calculated:

$$\bar{d}' = \frac{\bar{x}}{8t_{\bar{d}}'^4}. \quad (35)$$

Note that with $t_{\bar{d}}' \geq t_{\bar{d}}$ this results in $\bar{d}' \leq \bar{d}$.

It can be verified that this same approach is valid for the calculation or recalculation of all time intervals. It is important to note that with each new calculation of $\bar{d}'$ its value must reduce. This guarantees that none of the bounds that were checked in earlier steps of the algorithm will be violated.

State-of-the-art motion controllers are equipped with a high-resolution floating point calculation unit, such that quantization effects are usually negligible. To check

this, Eq. (33) can be used to calculate the obtained position in case of a quantized $\bar{d}'$. The difference between desired position and calculated position can be accounted for by means of a correction signal on the position trajectory. Typically in a digital system, the position signal is also quantized (usually in encoder increments). The correction signal can then be implemented by adding some increments to the position reference at each sampling instance during the trajectory. In relevant cases only one or two correction increments at each sampling instance should be sufficient (to prevent deterioration of the feedforward control). In case larger corrections are necessary, the accuracy and/or sampling rate of the motion control hardware should be increased.

6.2. Synchronization of profiles

Consider the continuous time implementation of the feedforward signal calculation as given in Fig. 5. Now, the generation of the derivative of jerk profile and the four integrators to obtain jerk, acceleration, velocity and position must be implemented in digital hardware. A straightforward approach is to replace the continuous time integrators by forward Euler discrete time integrators as indicated in Fig. 7. Clearly, all required profiles are thus calculated. However, due to the zero-order hold effect, each of the four integrators introduces a specific delay time. To fix this effect, the higher-order profiles can be delayed individually such that the symmetry of the complete set of profiles is restored. All this can be seen in Fig. 8, in which the discrete time profiles are compared with the corresponding continuous time profiles. The derivative of jerk profile must be delayed with $2 \times T_s$, the jerk profile with $1.5 \times T_s$, the acceleration profile with $1 \times T_s$ and finally the velocity profile with $0.5 \times T_s$. To obtain a delay of $0.5 \times T_s$ when sampling with T_s the average value is taken from the current and previous amplitude of the considered profile. This operation appears to work very well, although the associated smoothing effect is undesirable. Note that in this case the synchronization introduces a total delay of 0.1 s ($2 \times T_s$), but in practice the sampling time will be much smaller.

6.3. Implementation of first-order filter

All required profiles for calculation of the feedforward signal are now available. The multiplication with

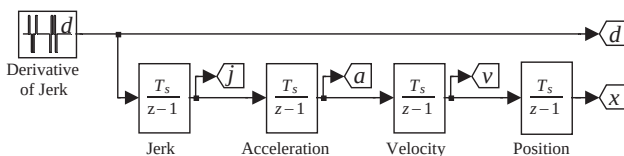


Fig. 7. Discrete time planner using forward Euler integrators.

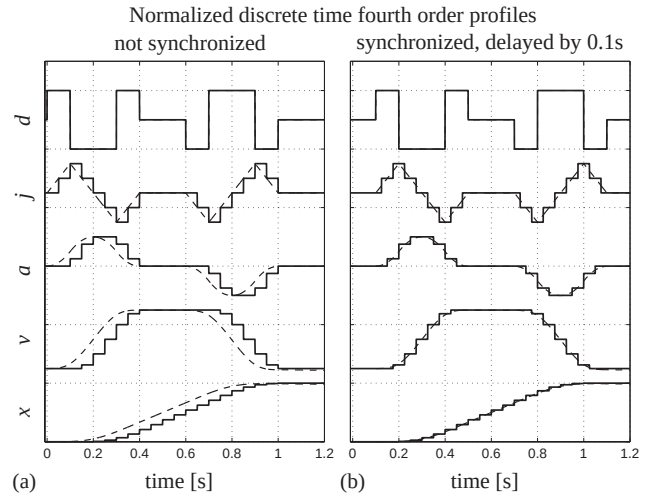


Fig. 8. Synchronization of discrete time profiles (dashed: continuous time).

factors q_1 to q_4 followed by summation as indicated in Fig. 5 is straightforward. The first-order filtering is less trivial, as it must also be transferred to the discrete time domain. A possible implementation that prevents problems with unwanted time delays and gives good results is to make use of the trapezoidal integration method. Fig. 9 shows the equivalence of the first-order filter in Fig. 5 with two possible implementations using the trapezoidal integration method. The first implementation has the inherent disadvantage that an algebraic loop occurs; in the second implementation this is prevented by re-calculating the loop. Equivalence can be checked by verifying that the discrete implementations both have the following transfer function:

$$\frac{y}{u} = \frac{(T_s/(2k_{12} + cT_s))(z + 1)}{z - (2k_{12} - cT_s)/(2k_{12} + cT_s)} \quad (36)$$

6.4. Calculation of reference trajectory

A final point on synchronization must be made with respect to the calculation of the reference trajectory that is used for feedback control. When applying the feedforward signal as calculated above, based on the synchronized profiles as illustrated in Fig. 8, the actual plant's response will be close to the ideal continuous time response with a delay of $2 \times T_s$. However, in order to compare this signal with the reference trajectory, it must be sampled with the same sampling frequency as is used for generation of the reference trajectory. The sample and hold device used for this will then introduce an average delay of exactly $0.5 \times T_s$.

Hence, to compensate for this further delay, it is necessary to also delay the reference trajectory with this same value. This additional delay can be implemented in the same manner as mentioned before: by taking the average between the current and previous reference

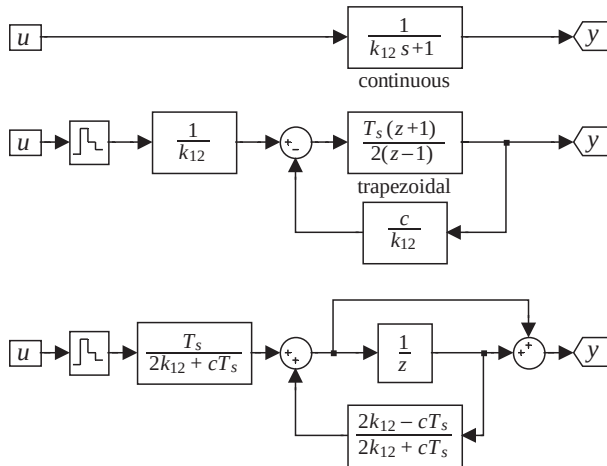


Fig. 9. Discrete implementation of first-order filter using the trapezoidal integration method.

trajectory value. The result of this will be that the control error will not be affected by sampling. The controller will only act on the effects of disturbances and on discrepancies between the actual plant and the modelled fourth-order behaviour.

Obviously, this is only true if the sampling frequency is sufficiently high, otherwise the momentary control error may deviate significantly from the average value. If this is the case, an increase in sampling frequency must be considered. Usually however, the sampling frequency is more significantly determined by the demands on stability and performance of the (digital) feedback controller.

7. Simulations and hardware-in-the-loop experiments

A theoretical motivation for the application of fourth-order feedforward as an extension of rigid body feedforward was already given in Section 3. Next, subsequent sections have shown that this approach is feasible in the sense of trajectory planning and (digital) feedforward implementation. This section will now show the effectiveness of fourth-order feedforward by considering the real-time implementation on the experimental motion system setup given in Fig. 10. This setup consists of a DC servomotor connected to a 500 lines per revolution encoder via a flexible axle, and can be controlled using Matlab/Simulink, real-time workshop and dSPACE equipment. The measured frequency response function given in Fig. 11 was used to determine a fourth-order model in accordance with Fig. 4, in which x_1 and x_2 should obviously be interpreted as rotations.

Now the main concern when considering model-based feedforward control is the occurrence of discrepancies between the behaviour of the actual motion system and the used model. This often motivates the use of rigid

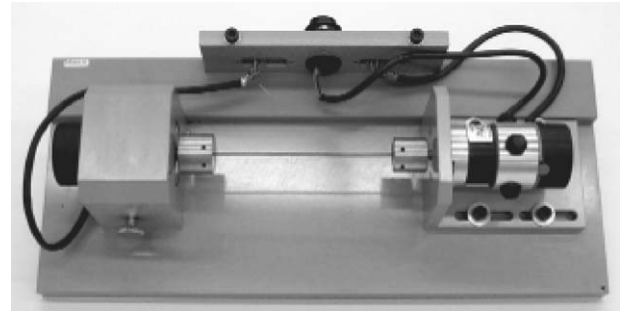


Fig. 10. Experimental motion system.

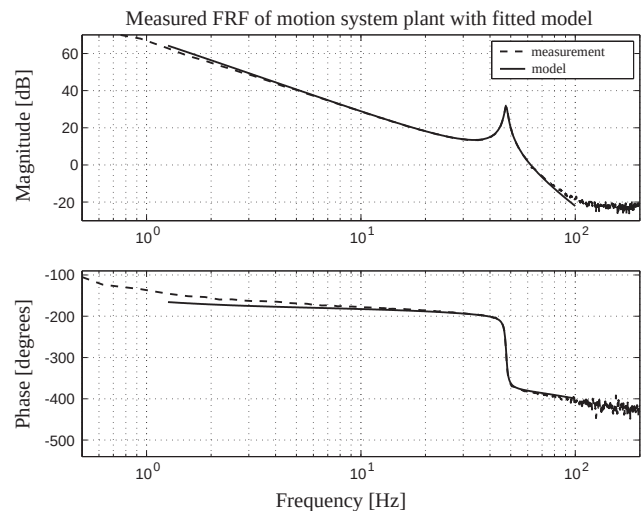


Fig. 11. Measured frequency response of motion system.

body feedforward: the total mass of the motion system is usually known within tight boundaries and a well-designed motion system will have limited friction and damping. Hence, the dependence of fourth-order feedforward on several additional physical parameters that may, or may not be constant should be investigated. The theoretical improvement of using fourth-order feedforward instead of rigid body feedforward must be robust against variations in these additional parameters: the mass ratio (division of mass in m_1 and m_2), the damping ratio (division of damping in k_1 and k_2), the spring stiffness c and the internal damping k_{12} (see Figs. 1 and 4). To demonstrate this, the fourth-order model derived from Fig. 11 is used to simulate the open-loop servo error response on the trajectory defined in Table 1. The simulations are performed in open loop to remove the effect of feedback control from the results. The motion system parameters and variations defined in Table 2 will be used, and all simulations are performed with the feedforward controller configuration of Fig. 5. When comparing with rigid body feedforward, the fourth-order trajectory will be applied, such that the smoothing effect of using a high-order trajectory is not accountable for the difference. It can easily be verified

Table 1
Fourth-order trajectory definition parameters

Parameter	Symbol	Value	Unit
Derivative of jerk	$\bar{d}$	10^8	rad/s^4
Jerk	$\bar{j}$	5×10^5	rad/s^3
Acceleration	$\bar{a}$	5000	rad/s^2
Velocity	$\bar{v}$	250	rad/s
Displacement	$\bar{x}$	100	rad

Table 2
Motion system parameters and variations

Parameter	Symbol	Value	Unit	Variation
Motor inertia	m_1	0.72×10^{-5}	kgm^2	$(0.605 \dots 0.835) \times 10^{-5}$
Load inertia	m_2	0.23×10^{-5}	kgm^2	$(0.345 \dots 0.115) \times 10^{-5}$
Spring stiffness	c	0.156	Nm	$\pm 33\%$
Internal damping	k_{12}	1.0×10^{-5}	Nms	$\pm 100\%$
Motor damping	k_1	1.0×10^{-5}	Nms	$(0.5 \dots 2.0) \times 10^{-5}$
Load damping	k_2	1.0×10^{-5}	Nms	$(2.0 \dots 0.5) \times 10^{-5}$

m_1 and m_2 are matched such that $m_1 + m_2$ is constant, the same holds for k_1 and k_2 .

that optimal rigid body feedforward is obtained by means of the configuration of Fig. 5 when setting $m_1 = 0.95 \times 10^{-5} \text{ kg m}^2$, $m_2 = 0$, $k_1 = 2 \times 10^{-5} \text{ N m s}$, $k_2 = 0$ and $k_{12} = 0$. With Eq. (6) this results in $q_1 = q_2 = 0$, $q_3 = m_1 c$ and $q_4 = k_1 c$. Furthermore, the first-order filter reverts to a constant gain of $1/c$ (see also Eq. (36)), and Eq. (7) reverts to Eq. (2). Note that the value of c becomes unimportant as it drops out of the calculations. The results are combined in Fig. 12. Note that in spite of the significant plant variations, the fourth-order feedforward controller performs at least twice as good.

The approach of choosing the parameters such that fourth-order feedforward reverts to rigid body feedforward suggests the use of third-order feedforward as an intermediate form by setting $q_1 = 0$. This would allow to implement a simpler third-order trajectory planner, with jerk being the highest bounded derivative. However, Fig. 13 shows that the effect of the jerk feedforward term q_2 is insignificant (and can hardly be improved by manual tuning of q_2). This shows that it is the derivative of jerk term q_1 that makes the difference. The fact that in practice it appears that third-order trajectory planning may lead to strong improvement of performance is therefore almost exclusively due to smoothing.

Fig. 14 shows that the servo error responses will not significantly deteriorate if fourth-order feedforward is implemented in discrete time. As an example, the motion system with minimal spring stiffness is considered. Note that the average delay between continuous time and discrete time signals equals the expected $2.5 \times T_s$.

Finally, the effectiveness of fourth-order feedforward is verified in real time. The measured frequency response function given in Fig. 11 was used to design a high-

Open loop 4th order FF response with plant variation

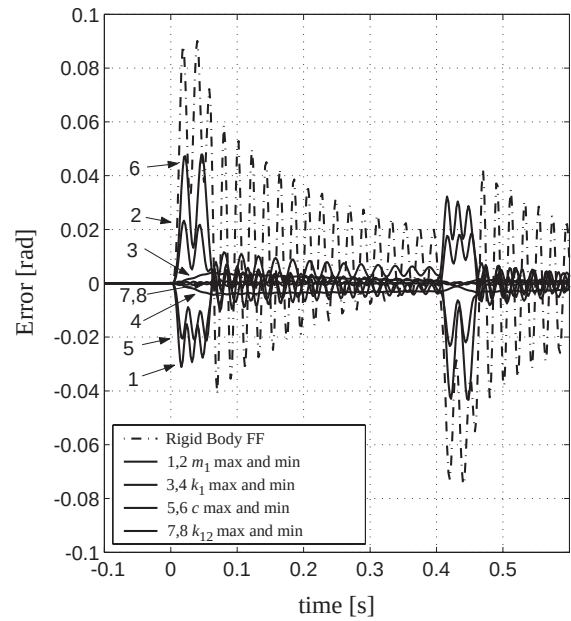


Fig. 12. Open-loop simulation results of fourth-order feedforward controller with plant variations, in comparison with optimally tuned rigid body feedforward.

Open loop response with optimal 2nd, 3rd and 4th order FF

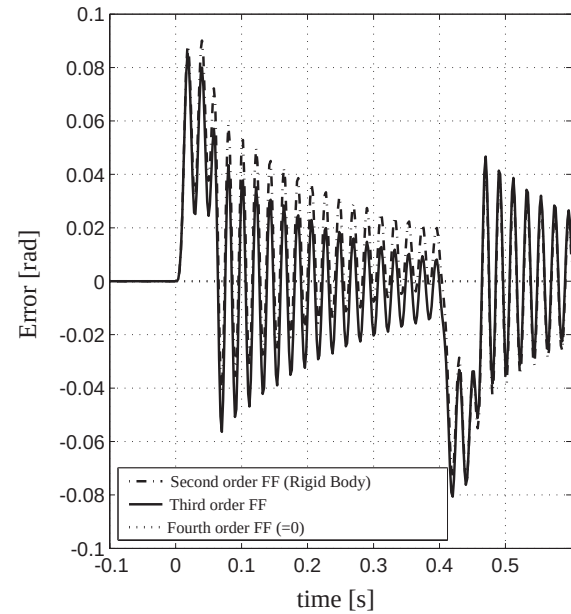


Fig. 13. Open-loop simulation results of optimal second-, third- and fourth-order feedforward controller with nominal plant.

performance feedback controller (with a bandwidth of about 20 Hz) that is implemented together with the discrete time version of the feedforward diagram in accordance with Fig. 5 at a sampling frequency of 500 Hz. In addition to this configuration, a Coulomb

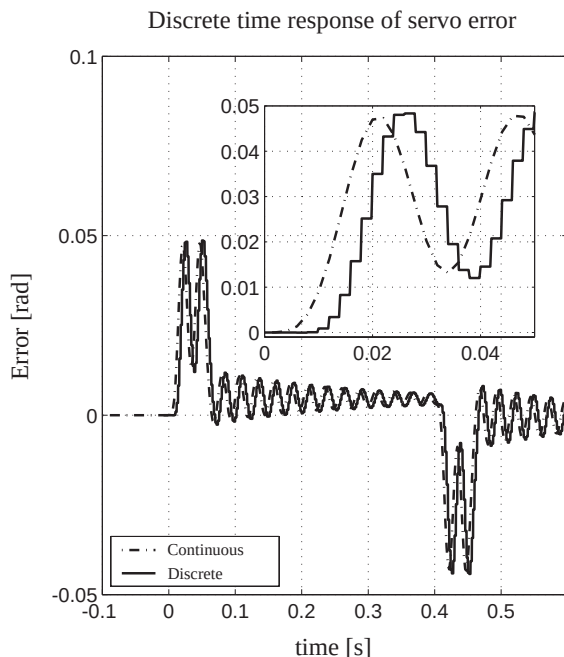


Fig. 14. Simulation results of open-loop discrete time fourth-order feedforward controller with minimal stiffness plant.

friction compensation scheme was implemented to account for the effect of dry friction in the setup. This approach is standard when using rigid body feedforward and can equally be used in combination with fourth-order feedforward. The servo error responses when using fourth-order feedforward versus rigid body feedforward are measured on the actual setup and given in Fig. 15 in comparison with simulations. Although the theoretical ‘zero-response’ of fourth-order feedforward is clearly too much to hope for in the actual experiment, a significant reduction of the servo error is indeed obtained. Note that the response with rigid body feedforward shows a significant relation with the jerk phases of the motion, which is also well predicted by the simulation, while the fourth-order feedforward response does not show this behaviour. The remaining response with fourth-order feedforward seems to be more related with the velocity profile, and may therefore be caused by unmodelled behaviour like, for instance, imperfect dry friction compensation or cogging forces. Also note that in the rigid body feedforward case, in comparison with the simulations, the damping of the resonance is significantly lower during the constant velocity phase and significantly higher at the end of the move. This can be explained by the effect of dry friction during measurement of the frequency response function of Fig. 11: the linear model fit will take some of the effect of dry friction into account as additional viscous damping. Finally, note that at the end of the move the rigid body feedforward has a significant settling behaviour, while the fourth-order feedforward shows

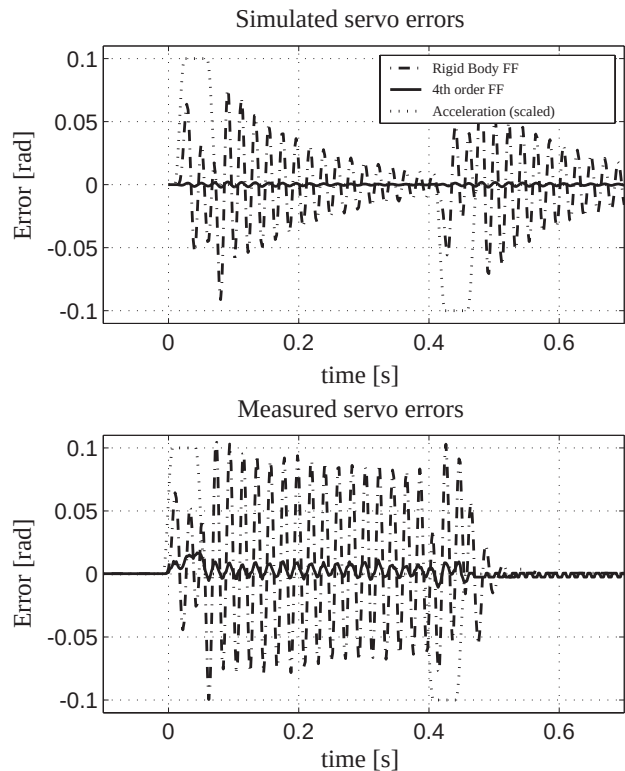


Fig. 15. Servo error responses simulated and measured on the experimental motion system.

errors in the order of the encoder resolution, and therefore effectively obtains zero-settling behaviour.

8. Conclusions

For high-performance motion control, especially for electromechanical motion systems, the usefulness of feedforward control is well known and often implemented. This paper shows that the popular simple feedforward scheme known as ‘mass feedforward’, ‘rigid body feedforward’ or ‘second-order feedforward’ can be extended to higher-order feedforward while still maintaining important practical properties like time-optimality, actuator effort limitation, reliability, implementability and accuracy. Furthermore, it is argued that the increase in complexity is manageable in state-of-the-art motion control hardware.

Third-order feedforward, which is increasingly applied in practice, appears to be mainly effective due to ‘smoothing’ of the trajectory, i.e. reduction of the high frequency content of the trajectory, such that feedback control can be more effective. The use of fourth-order feedforward for high-performance electromechanical motion systems is motivated by considering an appropriate model and supported by simulations and experimental results. Apart from the mentioned smoothing

effect, the feedforward of the derivative of jerk profile appears to result in a significant performance improvement. This is even more remarkable when considering that, for relevant cases, the calculated feedforward force is hardly affected.

A high-level algorithm is given to calculate higher-order trajectories for point-to-point moves; other motion commands, like speed change operations, can be derived from this. For fourth-order trajectory planning, the details of the algorithm are worked out, resulting in a practical, reliable and accurate algorithm.

Further implementation issues, like discrete time calculations, quantization effects and synchronization, are explicitly addressed. The trajectory planning algorithms can be implemented such that switching times are exactly synchronized with sampling instances. A digital implementation is suggested that takes care of the synchronization of the various profiles with each other and the position trajectory, and also with the measured position. It is shown that deterioration of the continuous time results due to sampling is small when applying a sufficient sampling rate. Experience shows that a sampling rate that is required for stable feedback control is also sufficient for feedforward control.

Simulation results show that the improvement obtained with fourth-order feedforward is not overly sensitive to variations in the parameters that are additional to the ‘classic’ parameters of rigid body feedforward (i.e. mass and viscous damping). Obviously, the feedforward performance will improve if the dynamical behaviour of the actual motion system is closer to that of the fourth-order model and said parameters are known within tighter bounds. An important advantage of the suggested implementation is the possibility to manually fine-tune the feedforward amplification factor for each profile (the q factors). This can be seen as a simple, direct extension of the well-known practice of fine-tuning rigid body feedforward.

Actual implementation has been performed on an experimental motion system setup. It is shown that the servo error with a well-tuned feedback and rigid body feedforward design can be improved upon significantly during all phases of a point-to-point move by means of fourth-order feedforward.

Future work in this area may address several issues. For instance, the matter of how to obtain optimal feedforward parameters for a given motion system is solved in this paper by fitting a fourth-order frequency response function on a measurement. Based on this, it should be possible to apply further techniques like on-line tuning, parameter estimation, learning and non-linearity compensation (e.g. scheduling). Another issue is to define fourth-order profiles for multiple axes motion systems, dealing with more complex paths and non-linear kinematic and dynamic behaviour.

A toolbox for Matlab and Simulink, containing the algorithms for obtaining third- and fourth-order profiles and several possibilities for using them in simulations and experiments is available on the internet: http://www.dct.tue.nl/New/Lambrechts/Advanced_Setpoints.zip

References

- Boerlage, M., Steinbuch, M., Lambrechts, P. F., & van de Wal, M. (2003). Model based feedforward for motion systems. *Proceedings of the IEEE international conference on control applications (CCA)*, Istanbul, Turkey (pp. 1158–1163).
- Bronstein, I. N., & Semendyayev, K. A. (1973). *A guide book to mathematics*. Frankfurt, Germany: Verlag Harri Deutsch ISBN 3-87144-095-7.
- Devasia, S. (2000). Robust inversion-based feedforward controllers for output tracking under plant uncertainty. *Proceedings of the American control conference*, Chicago, IL, USA (pp. 497–502).
- Dijkstra, B. G., Rambaratsingh, N. J., Scherer, C., Bosgra, O. H., Steinbuch, M., & Kerssemakers, S. (2000). Input design for optimal discrete-time point-to-point motion of an industrial XY positioning table. *Proceedings of the 39th IEEE conference on decision and control*, Sydney, Australia (pp. 901–906).
- Hunt, L., Meyer, G., & Su, R. (1996). Noncausal inverses for linear systems. *IEEE Transactions on Automatic Control*, AC-41(4), 608–611.
- Meckl, P. H., Arestides, P. B., & Woods, M. C. (1998). Optimized S-curve motion profiles for minimum residual vibration. *Proceedings of the American control conference*, Philadelphia, PA, USA (pp. 2627–2631).
- Murphy, B. R., & Watanabe, I. (1992). Digital shaping filters for reducing machine vibration. *IEEE Transactions on Robotics and Automation*, 8(2), 285–289.
- Paganini, F., & Giusto, A. (1997). Robust synthesis of dynamic prefilters. *Proceedings of the American control conference*, Albuquerque, NM, USA (pp. 1314–1318).
- Park, H. S., Chang, P. H., & Lee, D. Y. (2001). Concurrent design of continuous zero phase error tracking controller and sinusoidal trajectory for improved tracking control. *Journal of Dynamic Systems, Measurement, and Control*, 123, 127–129.
- Roover, D. (1997). *Motion control of a wafer stage*. The Netherlands: Delft University Press ISBN 90-407-1562-9.
- Roover, D., & Sperling, F. (1997). Point-to-point control of a high accuracy positioning mechanism. *Proceedings of the American control conference 1997*, Albuquerque, NM, USA (pp. 1350–1354).
- Singer, N., Singhose, W., & Seering, W. (1999). Comparison of filtering methods for reducing residual vibration. *European Journal of Control*, 5, 208–218.
- Steinbuch, M., & Norg, M. L. (1998). Advanced motion control: An industrial perspective. *European Journal of Control*, 4, 278–293.
- Tomizuka, M. (1987). Zero phase error tracking algorithm for digital control. *Journal of Dynamic Systems, Measurement, and Control*, 109, 65–68.
- Torfs, D. E., Swevers, J., & De Schutter, J. (1991). Quasi-perfect tracking control of non-minimal phase systems. *Proceedings of the 30th conference on decision and control*, Brighton, UK (pp. 241–244).
- Torfs, D. E., Vuerinckx, R., Swevers, J., & Schoukens, J. (1998). Comparison of two feedforward design methods aiming at accurate trajectory tracking of the end point of a flexible robot arm. *IEEE Transactions on Control Systems Technology*, 6(1), 1–14.